

TESTBENCH ARCHITECTURE FOR AXI TO AHB BRIDGE VERIFICATION USING UVM

1. Introduction

The AXI to AHB Bridge is used to connect systems operating on the AXI protocol with peripherals or subsystems that communicate using the AHB protocol. Since both protocols have different architectures and transaction mechanisms, a bridge is required to translate AXI transactions into equivalent AHB transactions while maintaining protocol compliance and data integrity.

To verify the functionality of the bridge, a UVM-based verification environment was developed. The testbench was designed to generate AXI transactions, monitor protocol activity on both sides of the bridge, and compare the observed behavior with the expected results. The environment supports write transactions, read transactions, reset verification, wait-state insertion, and error-response testing.

The overall goal of the verification environment is to ensure that every AXI transaction is correctly translated into an AHB transaction and that the responses generated by the AHB side are properly reflected back to the AXI side.

2. Testbench Architecture Overview

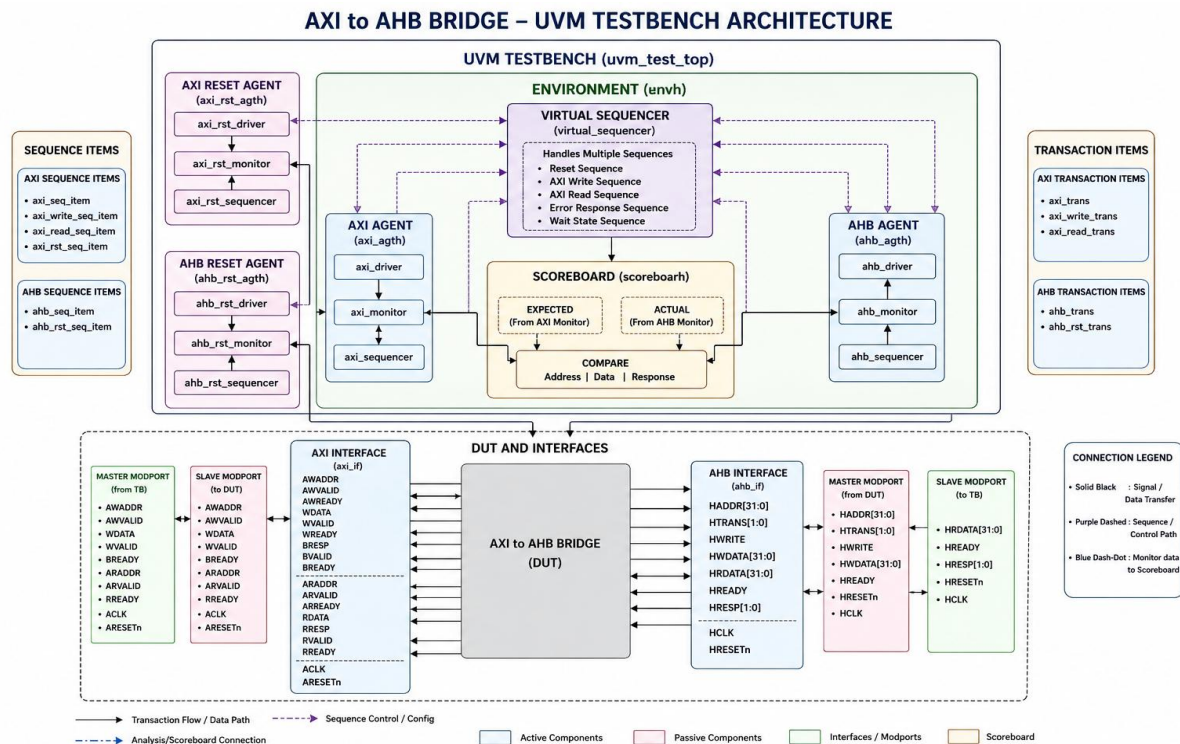
The verification environment follows a layered UVM architecture. The test controls the complete verification flow by configuring and running different sequences. The environment acts as the top-level container that instantiates all agents, the scoreboard, and the virtual sequencer.

The major components of the testbench are:

- AXI Agent
- AHB Agent
- AXI Reset Agent
- AHB Reset Agent
- Virtual Sequencer

- Scoreboard
- AXI to AHB Bridge DUT

The AXI Agent represents the master side of the system and is responsible for generating AXI traffic. The AHB Agent represents the slave side and generates different response scenarios. Transactions captured from both interfaces are sent to the scoreboard for checking.



3. AXI Agent

The AXI Agent is responsible for generating and monitoring AXI protocol activity. It operates as an active agent and consists of a driver, sequencer, and monitor.

AXI Driver

The AXI Driver receives sequence items from the sequencer and drives the corresponding AXI signals onto the interface. It handles the write address, write data, read address, and response channels according to the AXI protocol specification.

AXI Sequencer

The sequencer controls the flow of transactions between sequences and the driver. It supplies sequence items to the driver and allows different test scenarios such as write operations, read operations, and burst transfers to be executed.

AXI Monitor

The monitor passively observes the AXI interface and reconstructs complete AXI transactions from signal-level activity. These transactions are then forwarded to the scoreboard through analysis ports.

The AXI Monitor also contains dedicated analysis ports for write data and read data transactions, which helps in detailed transaction-level checking.

4. AHB Agent

The AHB Agent models the slave side behavior of the bridge. It is responsible for monitoring AHB transactions and generating different response conditions.

AHB Driver

The AHB Driver generates slave-side responses and controls the behavior of the AHB interface. It can generate normal responses, wait states, and error responses depending on the test scenario being executed.

AHB Sequencer

The AHB Sequencer provides transaction information to the driver and enables different response scenarios to be created for verification.

AHB Monitor

The AHB Monitor observes the AHB interface and captures address, control, data, and response information. Captured transactions are sent to the scoreboard for comparison against expected results generated from the AXI side.

5. Reset Agents

To verify reset functionality independently, separate reset agents were developed for both AXI and AHB protocols.

AXI Reset Agent

The AXI Reset Agent contains a driver, sequencer, and monitor. It generates reset sequences and verifies that the DUT correctly initializes when reset is asserted.

AHB Reset Agent

The AHB Reset Agent performs a similar function for the AHB interface. It generates HRESETn activity and monitors reset behavior during simulation.

Using dedicated reset agents improves testbench modularity and makes reset verification independent of normal transaction testing.

6. Virtual Sequencer

The Virtual Sequencer is used to coordinate multiple sequencers present in the environment.

It provides centralized control over:

- AXI Sequencer
- AHB Sequencer
- AXI Reset Sequencer
- AHB Reset Sequencer

The Virtual Sequencer simplifies the execution of complex scenarios where multiple protocol agents need to operate simultaneously. For example, reset sequences can be coordinated with AXI and AHB transaction sequences through a single control point.

7. Scoreboard

The scoreboard is the most important checking component of the verification environment.

Transactions captured by the AXI and AHB monitors are forwarded to the scoreboard through analysis ports and analysis FIFOs. The scoreboard receives the expected transaction information from the AXI side and compares it against the actual transaction observed on the AHB side.

The scoreboard performs the following checks:

- Address comparison
- Data comparison
- Response comparison

Through these checks, it verifies that the bridge correctly translates AXI transactions into equivalent AHB transactions without any loss of information.

8. Working of the Verification Environment

The overall verification flow is as follows:

1. A test starts execution and configures the environment.
2. The AXI Sequencer generates transaction sequence items.
3. The AXI Driver drives these transactions onto the AXI interface.
4. The AXI to AHB Bridge converts the AXI transaction into an AHB transaction.
5. The AHB Agent generates corresponding slave responses.
6. AXI and AHB Monitors capture protocol activity.
7. Captured transactions are forwarded to the scoreboard.
8. The scoreboard compares expected and actual transactions and reports the result.

This flow ensures complete end-to-end verification of the bridge functionality.

9. Conclusion

A complete UVM-based verification environment was successfully developed for the AXI to AHB Bridge. The architecture provides a structured and reusable framework for verifying protocol conversion between AXI and AHB interfaces. Through the use of protocol-specific agents, reset agents, a virtual sequencer, and a centralized scoreboard, the environment is capable of verifying write transactions, read transactions, reset functionality, wait-state insertion, and error handling scenarios. The modular nature of the testbench also allows future enhancements such as functional coverage and assertion-based verification to be added easily.